Ejercicios: Tema 04

1. Explica qué es un paradigma de programación. ¿Qué diferencia fundamental existe entre la forma de organizar un programa mediante el paradigma imperativo y mediante el paradigma orientado a objetos?
2. Explica qué limitaciones puede presentar la programación estructurada cuando un programa debe gestionar un número elevado de entidades que contienen información y operaciones propias. Utiliza como ejemplo una aplicación que gestione los datos de varios alumnos.
3. Indica qué se entiende por objeto en el paradigma orientado a objetos. Explica los conceptos de estado, comportamiento e identidad y pon un ejemplo de cada uno de ellos.
4. Considera una aplicación que gestiona los datos de varios vehículos. Indica qué características formarían parte del estado de un vehículo y qué operaciones podrían formar parte de su comportamiento. ¿Qué permitiría distinguir a un vehículo de los demás?
5. Explica qué es la abstracción y por qué resulta necesaria al diseñar una aplicación. Utiliza como ejemplo una clase que represente una cuenta bancaria: ¿qué características de una cuenta real considerarías relevantes para la aplicación?
6. Explica qué significa encapsular la información de un objeto. ¿Por qué es conveniente que los demás elementos del programa no tengan que conocer cómo se almacenan internamente sus datos?
7. Indica qué diferencia existe entre una clase y un objeto. Utiliza la analogía de una plantilla y los elementos construidos a partir de ella para explicar la relación entre ambos conceptos.
8. Considera una clase `Coche` que representa vehículos. Indica qué elementos podrían formar parte de su definición y cuáles serían características particulares de cada objeto creado a partir de ella.
9. Explica qué significa que un objeto sea una instancia de una clase. ¿Puede una misma clase dar lugar a varios objetos? ¿Pueden tener esos objetos valores diferentes para sus atributos?
10. Explica qué se entiende por interfaz pública de una clase y qué relación tiene con la encapsulación. ¿Por qué es conveniente que una clase ofrezca operaciones para utilizar sus objetos en lugar de permitir que otros elementos del programa manipulen directamente su representación interna?
11. Indica qué diferencia existe entre una variable de tipo primitivo y una variable de tipo referencia. Utiliza los siguientes ejemplos para explicar qué almacena cada variable:

    ```
    int edad = 20;
    String nombre = "Ana";
    ```

12. Explica qué significa que una variable de referencia almacene la referencia a un objeto. ¿La variable contiene todos los datos del objeto? Razona la respuesta.

13. Considera las siguientes declaraciones:

```
String nombre1 = new String("Java");
String nombre2 = nombre1;
```

14. Representa gráficamente la situación de las variables y los objetos en memoria. ¿Cuántos objetos se han creado? ¿Qué relación existe entre `nombre1` y `nombre2`?
15. Explica qué es un alias de referencia. ¿Qué consecuencias puede tener que dos variables hagan referencia al mismo objeto?
16. Considera el siguiente fragmento:

```
String texto1 = new String("Programación");
String texto2 = texto1;
texto2 = new String("Java");
```

17. Representa la situación de la memoria después de ejecutar cada sentencia. Indica qué ocurre con las referencias y qué sucede con el primer objeto cuando deja de estar referenciado.
18. Explica qué significa que una variable de referencia tenga el valor `null`. ¿Se ha creado algún objeto al declarar una variable e inicializarla con dicho valor?
19. Indica qué problema presenta el siguiente fragmento y en qué momento se produce:

```
String nombre = null;
System.out.println(nombre.length());
```

20. Explica qué función desempeña el operador `new` cuando se crea un objeto. ¿Qué relación existe entre este operador, la reserva de memoria y la ejecución del constructor?
21. Explica las diferencias entre Stack y Heap. Indica qué tipo de información se almacena habitualmente en cada una de estas zonas de memoria durante la ejecución de un programa Java.
22. Considera el siguiente método:

```
public static void mostrar() {
    int numero = 10;
    String texto = new String("Hola");
}
```

23. Indica qué elementos se almacenan en Stack y cuáles en Heap durante la ejecución del método. ¿Qué sucede con ellos cuando finaliza la ejecución?
24. Explica qué función desempeña el recolector de basura de Java. ¿En qué situación puede considerarse que un objeto está disponible para ser eliminado?
25. Considera las siguientes declaraciones:

```
int numero = 10;
String nombre = "Ana";
```

26. Explica qué diferencia existe entre los atributos de una clase y las variables locales de sus métodos. ¿Qué ámbito y duración tiene cada uno de ellos?
27. Diseña una clase `Punto` que permita representar un punto del plano mediante dos coordenadas enteras, `x` e `y`. Incluye los atributos necesarios y los métodos que permitan consultar sus coordenadas y modificar su posición.
28. Diseña una clase `Rectangulo` que permita representar rectángulos mediante su anchura y altura. Incluye los atributos necesarios y métodos para calcular su superficie y perímetro, así como para consultar sus dimensiones.
29. Implementa en Java la clase `CuentaBancaria`. Cada cuenta tendrá un titular y un saldo. Define los atributos y los métodos necesarios para consultar el saldo, ingresar dinero y retirar dinero. El saldo no podrá ser negativo y no se permitirá retirar una cantidad superior al saldo disponible.
30. Explica qué diferencia existe entre un método observador y un método transformador. Clasifica los métodos que has definido en los ejercicios anteriores según esta distinción.
31. Considera el siguiente método:

```
public void ingresar(double cantidad) {
    double nuevoSaldo = saldo + cantidad;
    saldo = nuevoSaldo;
}
```

32. Identifica los atributos, parámetros y variables locales que aparecen en el método. Explica qué ámbito tiene cada uno y en qué momento deja de existir la variable `nuevoSaldo`.
33. Explica cómo se transmiten los argumentos a los parámetros de un método en Java. ¿Qué significa que Java utiliza el paso de parámetros por valor?
34. Considera el siguiente fragmento:

```
public void modificar(int numero) {
    numero = 50;
}

public static void main(String[] args) {
    int valor = 10;
    modificar(valor);
    System.out.println(valor);
}
```

35. Indica qué valor se muestra por pantalla y explica por qué la modificación del parámetro no altera la variable utilizada como argumento.
36. Considera las siguientes clases y el método que modifica una referencia:

```
class Persona {
    String nombre;
}

public static void cambiar(Persona persona) {
    persona.nombre = "Luis";
}
```

37. Explica qué sucede si se llama al método `cambiar` pasando como argumento una referencia a un objeto `Persona`. ¿Se modifica el objeto original? Razona la respuesta teniendo en cuenta el paso por valor de las referencias.
38. Define una clase `Alumno` con los atributos necesarios para almacenar el nombre y la nota de un alumno. Incluye métodos para consultar ambos datos, modificar la nota y determinar si el alumno ha aprobado.
39. Explica qué es un constructor y cuál es su finalidad. ¿Qué diferencias presenta respecto a un método ordinario de una clase?
40. Escribe una clase `Producto` que almacene un código, una descripción y un precio. Define un constructor que permita inicializar los tres atributos cuando se crea un objeto.
41. Diseña una clase `Libro` con título, autor y número de páginas. Incluye un constructor que inicialice todos los atributos y métodos observadores que permitan consultar sus valores.
42. Explica qué se entiende por constructor por defecto en Java. ¿En qué circunstancias proporciona el compilador uno automáticamente? ¿Qué sucede cuando se declara un constructor en la clase?
43. Modifica la clase `Libro` para que permita crear objetos de dos formas: indicando todos sus datos o indicando únicamente el título y el autor. En este último caso, el número de páginas tendrá inicialmente el valor cero.
44. Explica qué es la sobrecarga de constructores. ¿Qué debe cambiar entre las declaraciones de dos constructores para que Java pueda distinguirlos?
45. Considera la siguiente clase:

```
public class Rectangulo {
    private double anchura;
    private double altura;

    public Rectangulo(double anchura, double altura) {
        anchura = anchura;
        altura = altura;
    }
}
```

46. Identifica el problema que presenta el constructor y corrígelo utilizando la palabra reservada `this`. Explica qué representa `this` dentro de un constructor.

47. Diseña una clase `Persona` con nombre y edad. Incluye dos constructores sobrecargados: uno que reciba ambos datos y otro que reciba únicamente el nombre, asignando en este caso una edad inicial de cero.
48. Explica qué es el encadenamiento de constructores y para qué puede utilizarse. ¿Qué ventaja tiene cuando varios constructores necesitan realizar parte de la misma inicialización?
49. Diseña una clase `Contador` que almacene un valor entero. Incluye métodos para incrementar, decrementar y consultar su valor. Define constructores que permitan crear un contador con valor inicial cero o con un valor inicial indicado.
50. Explica qué diferencia existe entre un miembro de instancia y un miembro estático de una clase. ¿Qué relación tiene cada uno con los objetos creados a partir de dicha clase?
51. Considera la siguiente clase:

```
public class Alumno {
    String nombre;
    static int numeroAlumnos;
}
```

52. Indica qué atributos pertenecen a cada objeto y cuál es compartido por todos los objetos de la clase. Explica qué sucede al crear varios objetos `Alumno`.
53. Modifica la clase `Alumno` para que cada vez que se cree un objeto se incremente automáticamente el atributo estático `numeroAlumnos`. Incluye un método estático que permita consultar cuántos alumnos se han creado.
54. Explica qué es una constante de clase y qué finalidad tienen las palabras reservadas `static` y `final` cuando se utilizan conjuntamente en la declaración de un atributo.
55. Define una clase `Matematicas` que contenga una constante de clase denominada `PI` y un método estático que calcule la longitud de una circunferencia a partir de su radio.
56. Explica por qué el método `main` de Java es estático. ¿Significa esto que todos los métodos y atributos de la clase que contiene `main` deben ser estáticos?
57. Escribe una clase `PruebaCuentaBancaria` que contenga el método `main`. Desde él, crea dos objetos de la clase `CuentaBancaria`, realiza diferentes ingresos y retiradas y muestra los saldos finales. Comprueba que cada cuenta mantiene su propio saldo.
58. Explica qué es una clase conductora y qué finalidad tiene cuando se desarrolla y prueba una aplicación. ¿Por qué resulta conveniente separar la clase que representa una entidad de la clase que contiene las instrucciones para probarla?
59. Explica qué se entiende por conductor y por resguardo en el contexto de las pruebas de software. ¿Qué función desempeña cada uno y en qué se diferencian?
60. Diseña una clase `Termometro` que almacene una temperatura en grados Celsius. Incluye métodos para consultar la temperatura, modificarla y obtener su equivalente en grados Fahrenheit. Escribe además una clase conductora que cree varios termómetros y pruebe sus operaciones.

61. Diseña una clase `Fecha` que almacene día, mes y año. Incluye constructores para inicializar sus datos, métodos observadores y un método que permita determinar si el año es bisiesto. Escribe una clase conductora que cree varias fechas y compruebe el funcionamiento de sus métodos.

Tema 4 | Problemas